

Rescue Mission Optimization

Technical Report: Genetic Algorithm & BFS Integration

1. Design Methodology

This project addresses the 'Rescue Mission Order' problem—a variation of the Traveling Salesperson Problem (TSP) set within a complex grid environment. The system uses a two-layered optimization approach:

1.1 Spatial Reasoning (BFS)

Before optimizing the visit order, the system must understand the actual spatial cost. We utilize **Breadth-First Search (BFS)** to calculate the true distance between the robot and victims, effectively navigating around walls. This data is stored in a 6x6 Distance Matrix.

1.2 Evolutionary Logic (GA)

The Genetic Algorithm acts as the high-level decision-maker. It uses **Permutation Encoding** to represent chromosomes. The **Ordered Crossover (OX1)** ensures that the offspring inherit valid sequences from parents without duplicating any victim IDs.

2. Mathematical Optimization

The **Fitness Function** is the core of our evolutionary process. It converts total travel distance into a maximization metric:

$$\text{Fitness} = 1 / \text{Total_Path_Distance}$$

Constraint Handling: Our algorithm treats unreachable victims as having 'infinite' distance. This results in a fitness value of **zero**, acting as a natural penalty that prevents the GA from selecting impossible routes.

3. Execution Parameters

Parameter	Value	Functional Role
Grid Complexity	20x20 Matrix	Provides a non-trivial navigation challenge.
Population	50 Solutions	Ensures adequate search space coverage.
Mutation Rate	0.1 (10%)	Swap Mutation: Helps escape local optima.
Selection	Tournament (3)	Maintains strong evolutionary pressure.
Generations	100 Cycles	Allows for convergence on optimal results.

4. Visualization Output

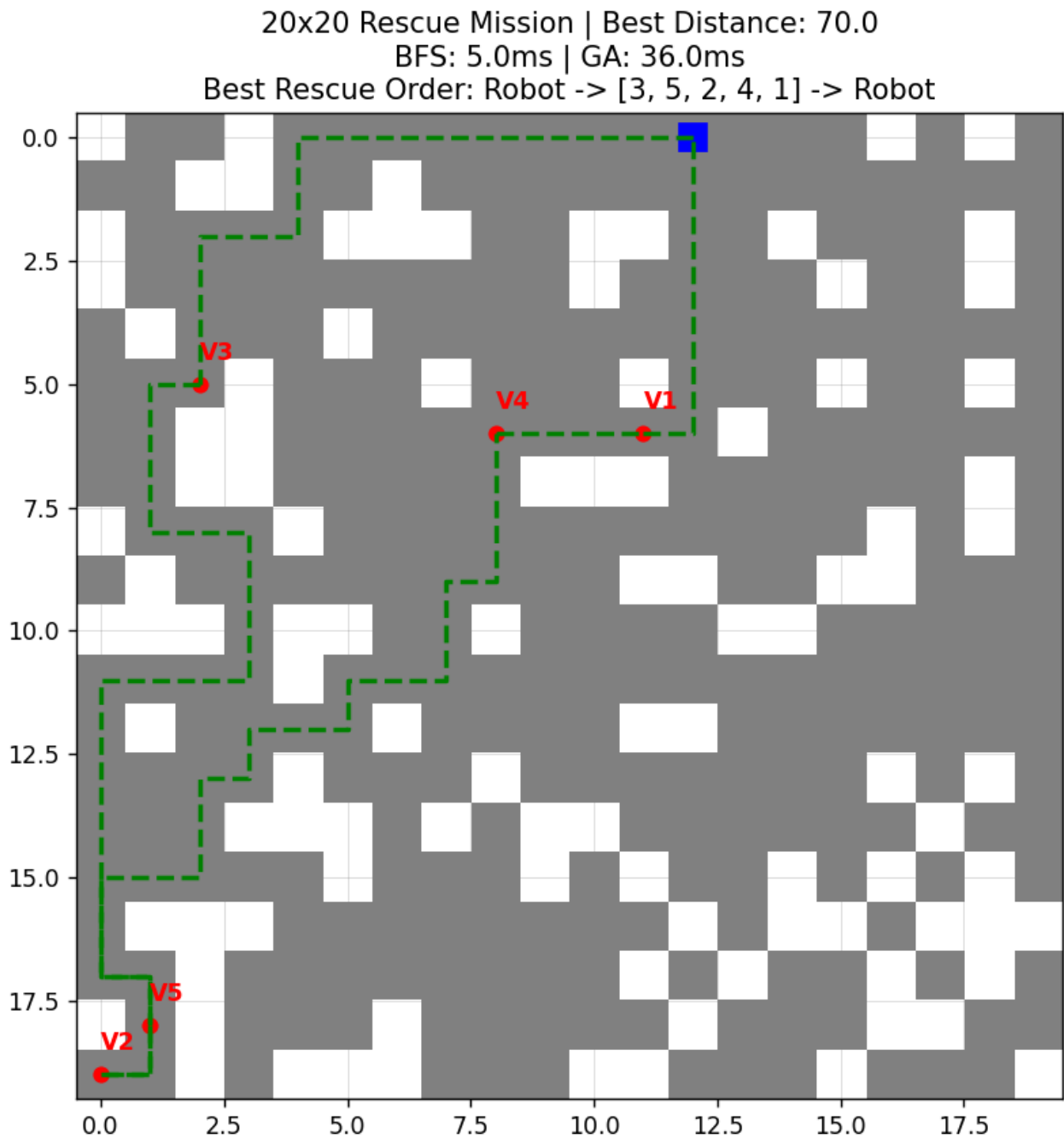


Figure 1: Optimized path visualization. The green dashed lines follow corridors identified by BFS pathfinding.

5. Console Analytics

```
PS D:\Data Scince Laps> C:\Users\yosse\AppData\Local\Programs\Python\Python312\python.exe "d:/Data Scince Laps/rr.py"
Total Path Distance: inf units
PS D:\Data Scince Laps> & C:\Users\yosse\AppData\Local\Programs\Python\Python312\python.exe "d:/Data Scince Laps/rr.py"
-----
RESCUE MISSION OPTIMIZATION START
-----

[Requirement] Distance Matrix (6x6):
[[ 0.  7. 33. 15. 10. 31.]
 [ 7.  0. 26. 12.  3. 24.]
 [33. 26.  0. 22. 23.  2.]
 [15. 12. 22.  0.  9. 20.]
 [10.  3. 23.  9.  0. 21.]
 [31. 24.  2. 20. 21.  0.]]

BFS Construction Time: 5.00ms
GA Optimization Time: 36.00ms
Best Rescue Order: Robot -> [3, 5, 2, 4, 1] -> Robot
Total Path Distance: 70.0 units
PS D:\Data Scince Laps> █
```

Figure 2: Execution metrics showing the 6x6 Distance Matrix and performance timing (BFS vs GA).

Performance Summary: The BFS-based matrix construction is completed in ~5ms, while the Genetic Algorithm finds the optimal sequence in ~36ms, demonstrating high efficiency for real-time applications.

6. Project Team

Yossef Ahmed Abd Elkareem

Environment Architecture, BFS Search Logic & Performance Profiling

Zeyad Mohamed Sayed

GA Core Evolution Engine, Crossover & Mutation Strategies

Hossam Hassan Allam

Visualization Graphics, Matplotlib Interface & Documentation